POLARIS New Member Onboarding

POLARIS-Ahmadi-OFFICIAL

Updated October 2026

Contents

Welcome to POLARIS

How POLARIS fits together

Repositories

Accounts and access

Prerequisites

Local setup

A tour of the app

Configuration cheat sheet

How we work

Deployment overview

Troubleshooting

Glossary

First-week checklist

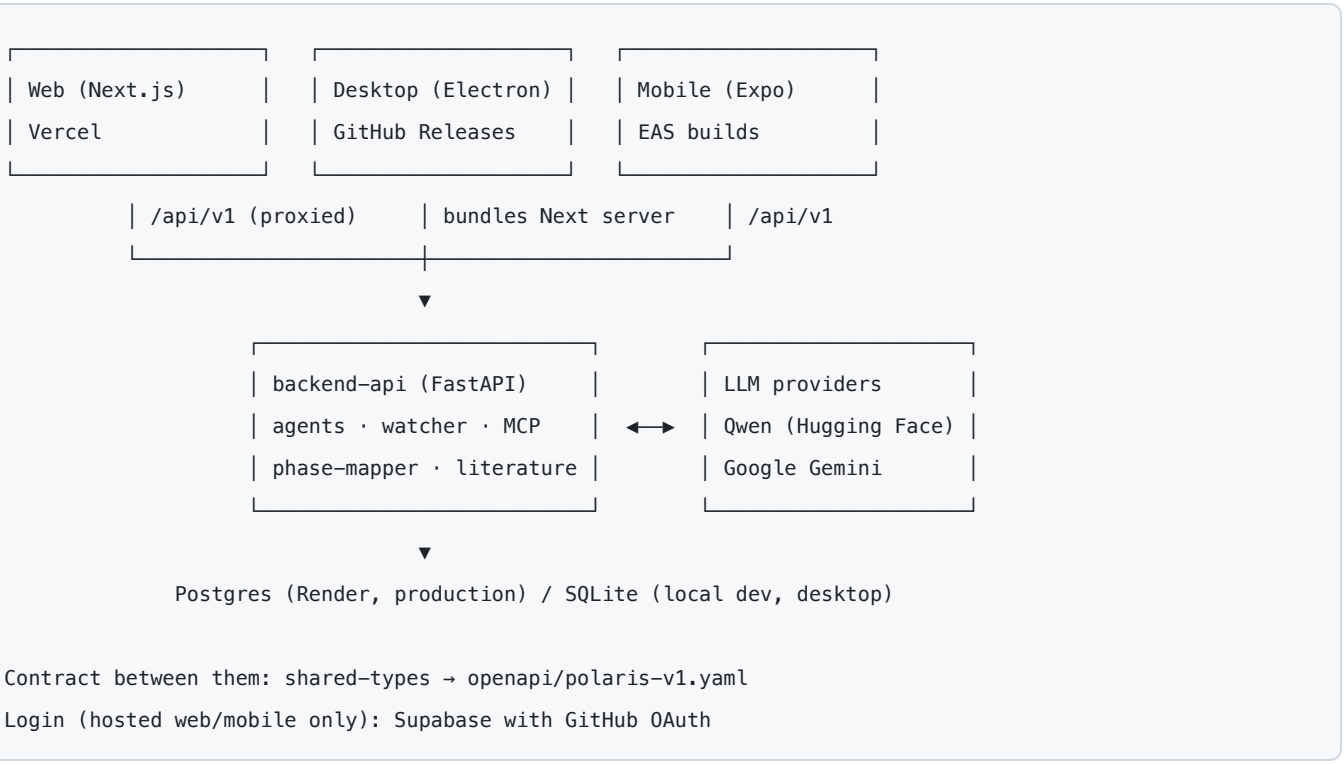
Getting help

Welcome to POLARIS

POLARIS is an AI research assistant for materials science. It helps researchers go from literature to hypothesis to experiment to analysis, with a focus on combinatorial **perovskite** libraries. Under the hood, a set of LLM-powered **agents** each handle one step of the research loop, and a web, desktop, and mobile app put them in front of researchers.

This guide gets you from zero to running the full stack locally and shipping your first change. Read it top to bottom once. Afterwards, use the First-week checklist and the per-repo READMEs as reference.

How POLARIS fits together



The agents

Agent	What it does
Literature	Searches and extracts papers, builds a knowledge graph, and supplies evidence to the other agents
Hypothesis	Multi-step reasoning (clarify, Socratic questions, tree-of-thought, synthesis) that produces a testable hypothesis

Agent	What it does
Experiment	Turns an experimental plan into worklists and well-plate layouts for automated mixing
Curve Fitting	LLM-assisted multi-peak fitting of spectral data
ML Models	Trains and runs ML models on experiment data
Analysis	Relates fit results back to the hypothesis and data, and decides whether more experiments are needed
Watcher	Watches a folder for new instrument data and routes it to the right agent
Phase Mapper	Predicts XRD phase labels and homogeneity from PL/absorbance/composition data and recommends which wells to measure next

Agents are chained in the **Workflow** page. The **MCP** page manages the Model Context Protocol orchestrator that serves literature tools to the hypothesis flow.

Repositories

All active code lives in the **POLARIS-Ahmadi-OFFICIAL** GitHub organization.

Repo	Stack	What lives there
backend-api	Python 3.12, FastAPI, LangGraph, SQLAlchemy	All agents, REST API (`/api/v1`), watcher, MCP orchestrator, database
web-frontend	Next.js, React, Tailwind, Electron	Web app and desktop app (the same codebase); also `npm run dev:stack`
mobile-development	Expo 56, React Native, NativeWind	iOS/Android app
shared-types	OpenAPI, TypeScript	The API contract (`openapi/polaris-v1.yaml`)

The original Streamlit monolith, CyberA183/polaris_ahmadi, was split into these repos at v1.0.0. Don't develop there.

Each repo has `docs/superpowers/specs/` (design docs) and `docs/superpowers/plans/` (implementation plans) for major features such as the phase mapper. Read the relevant spec before changing a feature. Old one-time docs live in `docs/archive/`.

Accounts and access

Ask a maintainer (an org admin) for:

- [ ] Membership in the **POLARIS-Ahmadi-OFFICIAL** GitHub org
- [ ] An LLM API key for local testing: a **Hugging Face** token (Qwen) or a **Google Gemini** key. Free tiers work but are rate-limited
- [ ] *(Only if you'll work on auth or deploys)* access to Supabase, Render, Vercel, and Expo/EAS

You do **not** need Supabase, Postgres, or any cloud account to run POLARIS locally.

Prerequisites

Tool	Version	Check
Git	any recent	`git --version`
Node.js	**22+**	`node -v`
Python	**3.12+**	`python3.12 --version`
Docker *(optional)*	for local Postgres	`docker --version`
Expo Go app *(optional)*	for mobile on a phone	App Store / Play Store

On macOS, install with Homebrew (`brew install node@22 python@3.12`). On Windows, use the official installers and run commands in PowerShell. Windows paths in the scripts are handled (`.venv\Scripts\python.exe`).

Local setup

1. Clone the repos side by side

`npm run dev:stack` expects the repos to be **sibling folders**:

```
mkdir POLARIS && cd POLARIS
git clone https://github.com/POLARIS-Ahmadi-OFFICIAL/backend-api.git
git clone https://github.com/POLARIS-Ahmadi-OFFICIAL/web-frontend.git
git clone https://github.com/POLARIS-Ahmadi-OFFICIAL/mobile-development.git
git clone https://github.com/POLARIS-Ahmadi-OFFICIAL/shared-types.git
```

2. Backend

```
cd backend-api
python3.12 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
cp .env.example .env
python init_db.py
uvicorn app.main:app --reload --port 8080
```

Check it: open http://127.0.0.1:8080/api/v1/health. You should see `{"status":"ok", ...}`. Browse every endpoint at http://127.0.0.1:8080/docs.

The default `.env` uses local SQLite and `AUTH_DISABLED=true`. Put your LLM key in `.env` (`HUGGINGFACE_API_KEY` or `GEMINI_API_KEY`), or add it later in the app under Settings → General.

3. Web

```
cd ../web-frontend
npm install
cp .env.example .env.local
npm run dev
```

`.env.example` points at the **production** API. To use your local backend, edit `.env.local` and set `NEXT_PUBLIC_API_URL=http://127.0.0.1:8080`, then restart `npm run dev`.

Open http://127.0.0.1:3000. On the login page, choose **Skip (dev only)**.

4. Mobile (optional)

```
cd ../mobile-development
npm install
npm start
```

Scan the QR code with Expo Go, or press `i` / `a` for a simulator. On a physical phone, set `expo.extra.apiUrl` in `app.json` to your computer's LAN IP (for example `http://192.168.1.20:8080`). Choose **Continue without auth (dev)**.

5. Everything at once

After steps 1–4 have been done once, start all three with prefixed logs:

```
cd web-frontend
npm run dev:stack       # API :8080 · web :3000 · Expo
```

6. Desktop (optional)

```
cd web-frontend
npm run dev:desktop     # Next dev server + Electron window
```

Installers are covered in `web-frontend/docs/DESKTOP-BUILD.md`.

A tour of the app

Page	Route	Use it to…
Home	`/home`	See an overview and jump into a task
Analytics	`/dashboard`	See agent usage, system performance, watcher status, and session stats
Workflow	`/workflow`	Chain agents into a pipeline and run it
Literature, Hypothesis, Experiment, Curve Fitting, ML Models, Analysis	`/agents/*`	Run one agent directly
Phase Mapper	`/phase-mapper`	Upload library data, view heatmaps and predictions, and pick the next wells for XRD
Watcher	`/tools/watcher`	Watch a data folder and auto-trigger agents
MCP	`/tools/mcp`	Manage the MCP orchestrator
Settings	`/settings`	Choose the LLM provider and model, and save API keys
History	`/history`	Review past sessions and runs

A good first exercise: set an LLM key in Settings, then run the **Hypothesis** agent with a simple research question and watch the requests in the backend terminal.

Configuration cheat sheet

Variable	Where	Purpose
`NEXT_PUBLIC_API_URL`	web `.env.local`	Backend base URL, **without** `/api/v1`
`NEXT_PUBLIC_SUPABASE_URL` / `_ANON_KEY`	web `.env.local`	Only for real GitHub login
`apiUrl`, `supabaseUrl`, `supabaseAnonKey`	mobile `app.json` → `expo.extra`	Same, for mobile
`DATABASE_URL`	backend `.env`	SQLite (default) or `postgresql+asyncpg://…`
`AUTH_DISABLED`	backend `.env`	`true` locally; `false` in production with Supabase
`LLM_PROVIDER`, `HUGGINGFACE_API_KEY`, `GEMINI_API_KEY`	backend `.env`	LLM access
`GEMINI_MIN_INTERVAL_SEC`, `HYPOTHESIS_*`	backend `.env`	Rate limiting and hypothesis speed (see backend README)
`CORS_ORIGINS`	backend `.env`	Browser origins allowed to call the API

Never commit `.env`, `.env.local`, API keys, or Supabase secrets. Only `.env.example` files belong in Git.

How we work

Branches and commits

- `main` is always releasable. Never push directly to it.
- Branch from `main` with a prefix: `feat/…`, `fix/…`, `docs/…`, `chore/…` (for example `fix/macos-gatekeeper-signing`).
- Use Conventional Commit messages, as in the existing history: `feat: add phase-mapper Downloads tab`, `fix: …`, `docs: …`, `chore: …`.

Pull requests

1. Push your branch and open a PR against `main`.
2. Describe **what** changed and **how you tested it**. Add screenshots for UI changes.
3. CI must pass (see below). Request a review from a maintainer.

4. Squash or merge once approved, then delete the branch.

What CI checks

Repo	On every push/PR to `main`	On a `v*` tag
backend-api	`pytest`, `ruff`, Docker build	n/a
web-frontend	`npm run lint`, `npm run build`	Builds macOS + Windows desktop installers into a GitHub Release
mobile-development	`tsc --noEmit`	EAS production build
shared-types	Spectral lint, build	`npm publish`

Run the same commands locally before pushing.

Changes that span repos

An API change usually touches three or four repos. Do them in this order:

1. **shared-types**: update `openapi/polaris-v1.yaml`, then `npm run lint:openapi && npm run build`.
2. **backend-api**: implement the route in `app/api/v1/` and add a test in `tests/`.
3. **web-frontend** and **mobile-development**: update `lib/` API clients and the UI.

Open one PR per repo and link them to each other in the descriptions.

Releases

Versions follow semver. Bump `version` in `package.json` (web/mobile/shared-types) or `pyproject.toml` (backend), merge, then tag:

```
git tag v1.4.0 && git push origin v1.4.0
```

Deployment overview

Piece	Host	Docs
API + Postgres	**Render** (`render.yaml`)	`backend-api/docs/DEPLOY.md`
Web	**Vercel**	`web-frontend/README.md`

Piece	Host	Docs
Desktop	**GitHub Releases** (CI on tag)	`web-frontend/docs/DESKTOP-BUILD.md`
Mobile	**Expo EAS**	`mobile-development/README.md`
Auth	**Supabase** (GitHub OAuth)	web and mobile READMEs

New members don't deploy. Ask a maintainer before changing anything in Render, Vercel, Supabase, or EAS.

Installing the macOS desktop app

The macOS build isn't notarized by Apple. After dragging POLARIS into Applications, if macOS says it "cannot be opened" or "is damaged", double-click **Fix POLARIS.command** in the DMG once. It clears the quarantine flag. This is expected.

Troubleshooting

Symptom	Fix
Web says the API is offline	Is the backend running on `:8080`? Does `NEXT_PUBLIC_API_URL` in `.env.local` match? Restart `npm run dev` after editing env files
`dev:stack` can't start the API	Create `backend-api/.venv` and `pip install -e ".[dev]"` first; repos must be sibling folders
`pip install` fails	Make sure you're on Python 3.12+ inside the venv (`python --version`)
429 / rate-limit errors from Gemini	Free tier. Wait ~60 s, keep `GEMINI_MIN_INTERVAL_SEC=6`, or switch to Qwen
Hypothesis agent is slow	Expected: it makes several sequential LLM calls. Keep `HYPOTHESIS_FAST_MODE=1`
401 errors from the API	Locally, set `AUTH_DISABLED=true` in `backend-api/.env`
Phone can't reach the API	Use your LAN IP in `app.json` `apiUrl`, not `127.0.0.1`; the Android emulator uses `10.0.2.2`

Symptom	Fix
CORS errors in the browser	Add your origin to `CORS_ORIGINS` in `backend-api/.env`

Glossary

Term	Meaning
Agent	An LLM-driven module in `backend-api/app/agents/` that handles one research step
Workflow	A user-defined chain of agents
MCP	Model Context Protocol, used here to serve literature tools to agents
Watcher	Filesystem watcher that triggers agents when instrument data lands in a folder
Phase Mapper	Pipeline that predicts XRD-derived phases for combinatorial perovskite libraries
Library / well	A combinatorial sample plate and one composition on it
PL / XRD	Photoluminescence / X-ray diffraction measurements
Socratic / tree-of-thought	Reasoning strategies the Hypothesis agent uses
Superpowers docs	`docs/superpowers/` specs and plans written before a big feature is built

First-week checklist

- [ ] Get org access and an LLM key
- [ ] Clone all four repos side by side
- [ ] Run backend, web, and (optionally) mobile locally; see `/api/v1/health` return ok
- [ ] Run `npm run dev:stack` successfully
- [ ] Generate a hypothesis end to end in the web app
- [ ] Skim `backend-api/app/agents/` and `web-frontend/app/(app)/`
- [ ] Read one spec in `docs/superpowers/specs/` (the phase-mapper spec is a good one)
- [ ] Run the tests: `pytest tests/ -q` and `npm run lint && npm run build`
- [ ] Open a small first PR (a docs fix is perfect) and get it through CI and review

Getting help

- Open a GitHub **issue** in the relevant repo for bugs and feature ideas.
- The apps have a built-in **feedback form** that emails bug and feature reports to the team.
- Ask a maintainer (an org admin) for access, keys, or review.